

## 0.76. Un déclencheur en (0, 0) : actif

Le A suit la croix depuis (10, 0) ; en arrivant en (0, 0), la variable **actif** passe de 0 à 1. C'est un déclencheur : il servira aux leçons 0.76.1 à 0.76.6.

### 1. Ce qu'il faut comprendre

**Le A part de (10, 0)** et suit la croix, avec sa position en direct, comme au 0.75. Mais cette fois, il ne se transforme pas : en arrivant en (0, 0), il **déclenche** quelque chose.

**Ce qui est nouveau ici : un drapeau, `actif`.** C'est une variable qui dit **oui ou non** : 0 = « pas encore », 1 = « le A est passé par (0, 0) ». On la voit à l'écran, en bas, à la colonne 10 : `000`, puis `001`.

**Une condition à trois morceaux :** `if (x == 0 && y == 0 && actif == 0)`. Le A est en (0, 0) **et** le drapeau vaut encore 0. La troisième partie fait que le bloc ne s'exécute **qu'une seule fois** : dès qu'`actif` vaut 1, elle devient fausse, même si le A reste sur la case.

**Pourquoi un drapeau :** les leçons suivantes font apparaître des lettres et les mettent en mouvement **à partir de ce moment-là**. Le drapeau retient que le moment est passé.

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1268 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          75 o ← la console, à l'allumage — la mise en route de la console
deplace_croix()    217 o ← main() — une fonction fournie par le langage
main()             84 o ← démarrage — ta fonction
VBlank             8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
AdresseCase        10 o ← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases       12 o ← effacer() — efface des cases
EcrireNombre       33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles       18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
LireManette        37 o ← bouton() — lit les huit boutons
Tuiles             704 o ← CopierTuiles — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
nombre()           x 3   30 o sur place, dans main() + EcrireNombre (33 o)
```

```

bouton()      × 4    20 o sur place, dans deplace_croix() + LireManette (37 o)
effacer()     × 1    15 o sur place, dans deplace_croix() + AdresseCase (10 o), EffacerCases (12 o)
poser()       × 1    13 o sur place, dans deplace_croix() + AdresseCase (10 o)
image()       × 1     3 o sur place, dans main() + AttendreImage (28 o)
images()      × 1     2 o sur place, dans deplace_croix()

Tes fonctions appelées :
deplace_croix() × 1    19 o sur place, dans main()

```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <deplace_croix> // une tuile qui suit la croix, case par case
#include <ALPHABET>      // les lettres de la police : ALPHABET[0] est le A
#include <nombre>        // écrit un nombre en chiffres

```

## 2. UNE VARIABLE — X

```
uint8_t x = 10; // le A part de (10, 0)
```

## 3. UNE VARIABLE — Y

```
uint8_t y = 0;
```

## 4. UNE VARIABLE — ACTIF

```
uint8_t actif = 0; // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

## 5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);
        nombre(10, 17, actif);                  // le drapeau : 000, puis 001

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
        }
    }
}

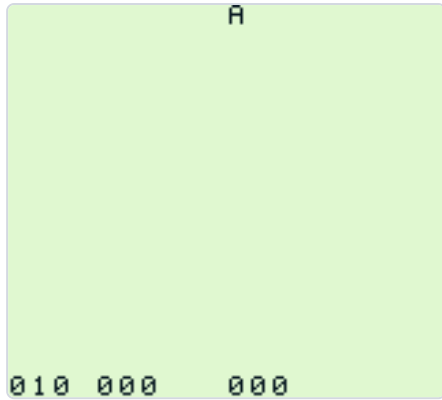
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Le A suit la croix ; en bas, sa position et le drapeau. En (0, 0), le drapeau passe à 001.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, actif vaut 0
- ✓ en (0, 0), actif vaut 1
- ✓ l'écran le montre : 001

Ensuite : **0.76.1. Un déclencheur en (0, 0) : le B apparaît au milieu** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++